

Machine Learning in Computational Biology

Assignment #2

Heart Disease Classification using Repeated Nested Cross-Validation

Deadline: Monday 11-05-2026 | Submission via e-class

Dataset

The dataset for this assignment is the **UCI Heart Disease Dataset (Cleveland subset)**, available by forking the repository: https://github.com/MLCB2026/MLCB_2026_Assignment_2

The dataset contains **242 patient records** and **14 attributes** (13 features + 1 target variable). The task is binary classification: predict whether a patient has heart disease based on the target variable `num`, where `num = 0` indicates no heart disease and `num = 1` indicates the presence of heart disease. All provided samples should be used in building your predictive models.

Objectives

The objectives of this assignment are to:

1. Explore and analyze the UCI Heart Disease dataset.
2. Implement a complete repeated Nested Cross-Validation (rnCV) machine learning pipeline following [object-oriented programming \(OOP\)](#) principles (required for this assignment).
3. Identify the best machine learning algorithm expected to correctly classify patients with and without heart disease based on the target variable, using the training and validation portions of the dataset.
4. Train a final model instance on all available data and save it for deployment.
5. Study which features drive the model's predictions.
6. Provide an interpretation of model prediction results using SHAP values.
7. Perform error analysis (Bonus task)

Task 1 — Exploratory Data Analysis

About the Dataset

The [Cleveland Heart Disease dataset](#) was collected at the Cleveland Clinic Foundation and is one of the most widely studied benchmarks in medical machine learning. Each record describes a patient using clinical measurements for the assessment of coronary artery disease.

The 13 features are described in the table below:

Feature	Type	Description
age	Continuous	Age in years
sex	Binary	Sex (1 = male, 0 = female)
cp	Categorical	Chest pain type (0–3): typical angina, atypical angina, non-anginal pain, asymptomatic
trestbps	Continuous	Resting blood pressure on admission (mm Hg)
chol	Continuous	Serum cholesterol (mg/dl)
fbs	Binary	Fasting blood sugar > 120 mg/dl (1 = true, 0 = false)
restecg	Categorical	Resting ECG results (0–2): normal, ST-T wave abnormality, left ventricular hypertrophy
thalach	Continuous	Maximum heart rate achieved during exercise
exang	Binary	Exercise-induced angina (1 = yes, 0 = no)
oldpeak	Continuous	ST depression induced by exercise relative to rest
slope	Categorical	Slope of the peak exercise ST segment (0–2)
ca	Ordinal	Number of major vessels (0–3) colored by fluoroscopy
thal	Categorical	Thalassemia type (1 = normal, 2 = fixed defect, 3 = reversible defect)

The dataset also contains missing values — you are expected to identify them and handle them appropriately.

About the Exploration

Create a Python notebook and perform a brief but concise exploratory data analysis. This should, at least, include:

1. Dataset Overview and Descriptive Statistics

Examine the shape and format of the dataset, column data types, duplicate rows and/or missing values, outliers, and class imbalance. Consider how missing values and class imbalance may affect your modeling choices.

2. Feature Assessment and Visualization

Explore the features of the dataset. Exploration may include box plots of their distributions and heatmaps, performing correlation analysis between features, and dimensionality reduction techniques such as PCA. Plots should be used to get a rudimentary sense of the feature space and

class separability. Pay attention to which features appear most informative and whether any preprocessing considerations arise from the data types present.

Task 2 — Intuition & Preparation for the nCV Implementation

Before implementing your nCV pipeline (or any class object), it is beneficial to grab a pen and paper. Imagine you have your components — the dataset and a series of untrained classifiers. Sketch out the abstraction of the nCV pipeline logic, showcasing the stages involved in terms of input, output, and role. Simple boxes (for pipeline steps) and arrows (for the flow of logic) with descriptive text are sufficient.

1. **Sketch the abstraction of one iteration of nCV.** Use 5 splits in the outer loop and 3 in the inner loop. Carefully trace the journey your data takes, and pay particular attention to where preprocessing and feature selection are applied, and [how to avoid data leakage](#).
2. For a more polished approach, you may use tools such as <https://app.diagrams.net/>.

Task 3 — Nested Cross-Validation Implementation

Step 3.1 — Create the Repeated Nested Cross-Validation Class

Create a Python file to construct a [Repeated Nested Cross-Validation](#) class equipped with essential methods to systematically train and estimate the generalization performance of classification algorithms on unseen data. The pipeline should accept as inputs, at least:

1. The dataset
2. A list of estimators
3. The respective hyperparameter spaces (defined by you) for each estimator
4. Number of rounds for the nCV ($R = 10$)
5. Number of outer fold loops ($N = 5$)
6. Number of inner fold loops ($K = 3$)

Keep in mind that:

- If the dataset is imbalanced, the folds you create should all reflect the dataset's overall class distribution.
- All preprocessing steps (imputation, encoding, scaling) and feature selection must be applied strictly within the pipeline — fitted on training data only and applied to validation/test data — to prevent data leakage.
- Your results need to be reproducible. Use specific seeds to avoid stochasticity.

Regarding the inner loop, choose a suitable metric for training and model selection. You can experiment with different optimization metrics for hyperparameter tuning and different methods for selecting the best trial (hyperparameter combination).

For classifier assessment, you should compute test-set statistics (including confidence intervals) in the outer nCV loop for metrics suited to the clinical context and the potential class imbalance of this dataset. These should include the Matthews Correlation Coefficient (MCC), Area Under the ROC

Curve (AUC), Balanced Accuracy (BA), F1 score, Recall, Specificity, Precision, Precision-Recall AUC (PRAUC), etc. Using multiple complementary metrics provides a clearer picture of the algorithm's performance.

Step 3.2 — Assess Generalization Performance and Select the Winner Algorithm

Import your nested cross-validation class into a new Python notebook. Using your full pipeline (including preprocessing), evaluate the following algorithms with default hyperparameters:

Logistic Regression (LR) with Elastic Net regularization, **Gaussian Naive Bayes (GNB)**, **Linear Discriminant Analysis (LDA)**, **Random Forests (RF)**, **LightGBM**, **XGBoost**, and **CatBoost**.

You should do this by disabling the inner loop (no hyperparameter tuning) and perform $R=10$ iterations of 5-fold cross validation. This serves as a fair baseline comparison across algorithms.

Next you need to estimate the generalization performance of the algorithms. Define and perform $R = 10$ iterations (rounds) of nCV (repeated nested CV, or rnCV). Use $N = 5$ folds for the outer loop and $K = 3$ for the inner loop.

You may use regularized versions of these algorithms, and you should define (with justification) your own hyperparameter space for each one of them (as per the previous assignment). Assess the classification algorithms using performance metrics aligned with the classifier's intended clinical purpose, and declare as "winner" the algorithm that achieves the best purpose-aligned balance of these metrics across 10 repetitions of nCV. This means considering the median metrics across all outer-loop test folds ($5\text{-fold nested CV} \times 10\text{ repetitions} = 50$ evaluation points per metric per algorithm). Fully justify your approach when comparing algorithms.

In cases where the median performance of two or more top algorithms is similar, use sound statistical practices — for example, considering the 95% confidence interval of the medians — in your winner selection.

Task 4 — Feature Selection

After identifying the winning algorithm, perform **model-agnostic feature selection** to identify the most informative subset of features. Modify your rnCV class to include feature selection on the outer loop. (Hint: Feature selection should be done using only on training folds to avoid data leakage!)

Re-run the rnCV pipeline using only the selected features and compare the results against the full-feature baseline established in Step 2 in Task 3.

The choice of feature selection method, the number of features retained, and the criteria used to make these decisions are left to you and should be justified.

Keep track of which features are consistently selected across folds and repetitions (stable features) and report the selection frequency per feature. Discuss your findings: does using a reduced feature set match, improve, or degrade performance? What does this suggest about the dataset and the winner model? Which are the most selected features and are they clinically meaningful?

Task 5 — Train the Final Model Instance

Step 5.1 — Select the Best Hyperparameters Using Simple CV

After identifying the winner algorithm using rnCV, perform a final 5-fold cross-validation (CV(5)) now using the whole dataset to determine the optimal set of hyperparameters for the final model instance to deploy.

Step 5.2 — Train the Final Model Instance

Train the final model instance using the optimal hyperparameters on all available data, and save it as a `.pkl` file in the `./models/` directory for deployment. The final model must be a complete pipeline ready to accept raw, unprocessed input and return predictions. We will use this model for the class competition, and it should be runnable.

Your final trained model will be evaluated on a **secret held-out test set** that has been withheld from students. The student whose model achieves the highest performance on this held-out set will automatically receive **bonus points** for winning the competition!

Step 5.3 — Final Model Interpretation using SHAP Values

Using the final trained model from Step 5.2, compute [SHAP \(Shapley Additive exPlanations\)](#) values to interpret the model's predictions. *SHAP values should be computed on the model trained on all data*, and **not** on intermediate models obtained during cross-validation.

[SHAP values](#) quantify in a model-agnostic manner the contribution of each feature to individual predictions, grounded in cooperative game theory.

Produce appropriate SHAP visualizations and use them to discuss which features most strongly drive the model's predictions. Reflect on whether these findings are consistent with the patterns observed during EDA and with established clinical knowledge about heart disease risk factors. The appropriate SHAP explainer will depend on the type of winner model — consider this carefully.

Bonus Task (Optional)

Bonus tasks are entirely optional and will not affect your base course grade. However, students who consistently accumulate bonus points across assignments will receive a course grade boost of up to one full mark (e.g., 7.2 may become 8). This extra effort will also be emphasized in any reference letters.

Error Analysis (+50 % bonus points)

Using the winning algorithm identified in the main part of the assignment, split the dataset into a training and a validation set, train the model on the training data, and evaluate its performance on the validation set.

Based on the validation predictions, perform a detailed error analysis with particular emphasis on misclassified samples, specifically false positives (patients incorrectly predicted to have heart disease) and false negatives (patients incorrectly predicted to be healthy). Compute SHAP (SHapley Additive exPlanations) values for the predictions on the validation set using the trained model, and use them to interpret the model's decision-making process.

Analyze the characteristics of misclassified patients by comparing them with correctly classified cases. As a starting point, examine the age distribution across false positives, false negatives, and correctly classified samples, and determine whether specific age groups are more prone to misclassification. Extend this analysis to other clinically relevant features and identify patterns associated with prediction errors. Furthermore, compare the SHAP value distributions of misclassified and correctly classified instances to determine which features contribute most strongly

to incorrect predictions and whether particular patient subgroups are systematically associated with errors.

Finally, produce a concise clinician-oriented report that explains how the model makes decisions, highlights the factors driving correct and incorrect predictions, identifies patient groups at higher risk of misclassification, and discusses the limitations and potential clinical risks of deploying such a model in practice.

Deliverables

By the deadline, submit a ZIP file named:

Assignment2_YourLastName_FirstName_ID.zip

The ZIP must include:

1. Repository (provide the link in your report)

The repository architecture should follow the same structure as the previous assignment: a folder for notebooks, a src folder containing your nested cross-validation class and any additional Python files, and a models directory to store your final model instance in .pkl format.

2. Technical Report (10-15 pages, 11pt, 1.15 line spacing)

A technical report in the form of a brief scientific paper, prepared using LibreOffice Writer, MS Word, or LaTeX. The report should contain the following sections:

1. **Abstract** — Concise summary (max 200 words) of the problem statement, results, and their impact.
2. **Introduction** — Problem statement, significance, related work, technical approach, and contributions.
3. **Materials and Methods** — Figure from Task 2, dataset description, preprocessing approach, nCV pipeline structure and stage descriptions.
4. **Results and Discussion** — EDA findings, algorithm comparison results and discussion. If any bonus tasks were attempted, report their results in a dedicated subsection.
5. **Conclusions** — Summary of main findings, limitations of the analysis, lessons learned, and suggested further research.
6. **References** — IEEE citation style, referencing relevant scientific articles, resources, and web pages.
7. **AI Usage Disclosure** — Specify which AI tools were used and for which tasks. If this section is absent, it will be taken to mean that no AI tools were used.

Hints and Tips

- If you build your nCV pipeline using scikit-learn directly, consider using **Optuna** (<https://optuna.org/>) for hyperparameter tuning in the inner loop. Up to 50 trials should suffice.
- To construct confidence intervals for non-parametric statistics such as the median, bootstrap resampling is a suitable approach. This can be implemented manually or using a library such as `scipy`.
- Many classifiers may produce warnings (e.g., convergence warnings) during training. Once you have verified that your code runs correctly, you can safely suppress these to keep your notebook readable.

Academic Integrity

An important note on AI usage: You are responsible for the code and report you deliver. Assignment solutions or reports that are copied and pasted from an LLM are distinguishable and may be subject to a follow-up oral examination to ensure fairness. You are doing yourselves a great disservice if you rely on LLMs to complete this assignment. Moreover, you are required to include an honest “AI Usage disclosure” statement at the end of your report, indicating if you used an LLM and how.

Good luck — start as soon as possible!